FinCore — Implementation Plan (Phases & Tickets)

Oct 22, 2026 · @Mohamed

How to use this plan

FinCore is built in 10 phases, each with numbered tickets and a tangible output; a phase is closed only when every item in its output list passes. Tickets are named P<phase>-<number>.

Size	Meaning
S	Up to half a day
M	1 to 2 days
L	3 to 5 days, split it if it grows

Three working rules apply to every ticket:

1. Write the failing test first for every invariant (balanced entries, legal transitions, exactly-once effects).
2. Do not start the next phase until the current phase's output list is fully green.
3. Record every decision as a short ADR (one page: context, decision, consequences).

Phase 1: Foundation & Payment API (weeks 1-2)

Goal: a merchant can be onboarded and can create and read payments safely, with money represented exactly.

ID	Ticket	Size	Done when
P1-1	Solution skeleton: Domain, Application, Infrastructure, Api, Workers, ProviderSimulator, Tests	S	Builds; Domain has zero project references

ID	Ticket	Size	Done when
P1-2	Money value object: long minor units, currency exponent, add/subtract, overflow checks	M	EGP + USD throws; EGP + EGP works
P1-3	Seed currencies EGP, USD, EUR, SAR with exponents	S	Unsupported currency returns 422 currency_unsupported
P1-4	Merchant, Customer, Payment aggregates with domain invariants and public prefixed ids (mer_, cus_, pay_)	M	Amount 0, negative or overflow cannot be constructed
P1-5	EF Core configurations, Money value converter, first migration, CHECK constraints and indexes	M	Direct SQL INSERT with AmountMinor = 0 is rejected
P1-6	POST /merchants (admin key): generate API key, store only SHA-256 hash and 8-char prefix	M	Raw key appears nowhere in DB or logs
P1-7	ApiKeyAuthenticationHandler	M	Missing, malformed, revoked or suspended key returns 401 ProblemDetails
P1-8	ExceptionHandlingMiddleware and CorrelationIdMiddleware	M	No stack traces leak; X-Correlation-Id echoed and in logs
P1-9	POST /customers and POST /payments with FluentValidation, integer-only amounts	M	Amount 10.5 is rejected; invalid requests write nothing
P1-10	GET /payments/{id} and GET /payments with cursor pagination	M	Foreign merchant gets 404; pages stable during inserts
P1-11	Swagger with API-key scheme; unit tests for Money and Payment	S	Domain tests run without a database; coverage of Money and Payment at 90% or more

Phase output

- Working solution skeleton with Swagger UI and migration scripts.
- Tables: Merchants, Customers, Payments.
- Authenticated API: POST /merchants, POST /customers, POST /payments, GET /payments/{id}, GET /payments.

- Unit-test project covering Money and Payment.

Phase 2: Idempotency & Lifecycle State Machine (weeks 3-4)

Goal: retries are safe, and a payment can only move along legal states.

ID	Ticket	Size	Done when
P2-1	Status enum and transition table in the Domain; Payment.Transition(newStatus, reason) is the only way to change status	M	No other code path assigns Status
P2-2	Exhaustive unit-test matrix: every from/to pair	M	Every illegal pair fails, every legal pair succeeds
P2-3	PaymentStatusHistory (insert-only), one row per transition with actor and correlation id	M	History matches transitions in chronological order
P2-4	CHECK constraint on status names; RowVersion concurrency mapped to 409	S	Direct SQL UPDATE to an illegal status is blocked
P2-5	IdempotencyKeys table, unique (MerchantId, Key), fingerprint = SHA-256 of canonical body + method + route	M	Same key from two merchants does not collide
P2-6	IdempotencyFilter: insert-first, replay with Idempotent-Replayed header, 409 in progress, 422 key reuse, 400 missing key	L	50 parallel identical requests create exactly one payment
P2-7	Failure handling: rollback frees the key; stale InProgress key can be retried after the lock timeout	M	No key is poisoned after a crash
P2-8	Endpoints: confirm, capture (amount not above authorized), cancel, history	M	Capture above authorized returns 422 capture_exceeds_authorized
P2-9	IdempotencyCleanupService purging keys older than 24 hours	S	Expired key is treated as new

ID	Ticket	Size	Done when
P2-10	Testcontainers SQL Server integration suite including race and concurrent-capture tests	L	Two parallel captures: one succeeds, one gets 409

Phase output

- Idempotent POST /payments with the full behaviour contract (replay, in progress, reuse, missing).
- PaymentStateMachine with a complete unit-test matrix.
- Tables: IdempotencyKeys, PaymentStatusHistory.
- Endpoints: confirm, capture, cancel, history.
- Integration test suite running against a real SQL Server container.

Phase 3: Double-Entry Ledger Core (weeks 5-6)

Goal: capturing a payment posts one balanced journal entry in the same transaction as the status change.

ID	Ticket	Size	Done when
P3-1	LedgerAccounts table and chart-of-accounts seeder (1000, 1100, 2000, 4000, 5000)	M	Accounts unique per code, currency and merchant
P3-2	Merchant onboarding hook creating the merchant payable account	S	New merchant has a 2000 account
P3-3	JournalEntries and JournalLines tables with CHECK constraints and unique (SourceType, SourceId, EntryType)	M	Raw SQL with debits 100 and credits 99 is rejected
P3-4	JournalEntryDraft and domain invariants: 2 or more lines, amount above 0, one currency, active accounts, balanced	M	Single-line, zero, negative and mixed-currency drafts rejected
P3-5	FeeCalculator: percentage plus fixed fee, round half up, remainder to fee revenue	M	For 1, 33, 99, 101, 99,999: fee + merchant share = gross

ID	Ticket	Size	Done when
P3-6	LedgerService.PostAsync as the only door into the ledger, atomic header and lines	L	No other code writes to ledger tables
P3-7	DB trigger checking line sums against the header (fallback: stored procedure with table-valued parameter)	M	Trigger fires on direct inserts
P3-8	Capture endpoint posts the entry in the same transaction as the status change	M	Forced ledger failure leaves payment Authorized
P3-9	Admin endpoints: create/list accounts, account statement, journal entry, trial balance	M	Trial balance debits equal credits
P3-10	FsCheck property test, 1,000-payment trial balance test, wiki page "how to read a journal entry"	M	Signed amounts sum to zero for any valid event sequence

Phase output

- Capture of 25,000 piasters produces exactly one entry: 25,000 debit to Provider Clearing, 24,375 credit to Merchant Payable, 625 credit to Fee Revenue.
- Tables: LedgerAccounts, JournalEntries, JournalLines.
- Admin ledger endpoints and a trial balance report that always nets to zero.
- Documented chart of accounts.

Phase 4: Ledger Immutability, Reversals & Balances (weeks 7-8)

Goal: history cannot be rewritten, refunds work as reversals, and balances stay correct under concurrency.

ID	Ticket	Size	Done when
P4-1	Append-only enforcement: DENY UPDATE/DELETE for the app login, INSTEAD OF triggers that THROW	M	UPDATE or DELETE fails even for a db_owner session

ID	Ticket	Size	Done when
P4-2	Sequence, PrevHash and EntryHash (SHA-256 chain); unique index on ReversesEntryId	M	Changing one old row breaks every later hash
P4-3	LedgerService.ReverseAsync with mirrored lines; admin reverse endpoint with mandatory reason	M	Reversing twice returns 409
P4-4	AccountBalances projection updated atomically inside the posting transaction	M	Projection equals the sum of lines after 10,000 operations
P4-5	Refunds table and RefundService: lock payment row, compute remaining refundable, post reversal, transition, one transaction	L	Refund of 100.00 of 250.00 moves exactly 10,000 piasters
P4-6	Refund endpoints with idempotency; statuses PartiallyRefunded and Refunded	M	Refund above remaining returns 422 refund_exceeds_captured
P4-7	Pending vs available balance and GET /merchants/me/balance	M	Pending + available equals Merchant Payable balance
P4-8	LedgerIntegrityVerifier, /admin/ledger/integrity JSON report, projection rebuild	M	Report names the first broken sequence number
P4-9	Concurrency tests: 100 parallel refunds of 5.00 EGP against a 250.00 payment	L	At most 50 succeed; total never exceeds captured amount

Phase output

- Anti-tamper SQL scripts: permission model and triggers.
- Tables: AccountBalances, Refunds; new columns Sequence, PrevHash, EntryHash on JournalEntries.
- Endpoints: refunds, merchant balance, admin reversal, integrity check.
- A JSON integrity report format listing broken links or mismatched balances.
- Refund policy documented: fee is non-refundable by default.

Phase 5: Provider Simulator & Timeouts (weeks 9-11)

Goal: handle an unreliable external provider, especially the case where the outcome is unknown.

ID	Ticket	Size	Done when
P5-1	Provider Simulator project: authorizations, capture, void, refunds, inquiry by reference, idempotent by key	L	Same reference never creates a second authorization
P5-2	Magic tokens (tok_success, tok_decline_insufficient_funds, tok_timeout, tok_error_500, tok_slow_5s, tok_success_no_response) and scenarios endpoint	M	Each token produces its documented behaviour
P5-3	IPaymentProviderClient, SimulatorProviderClient and the anti-corruption mapping of decline codes with a retryable flag	M	Rest of the system never sees raw provider DTOs
P5-4	Polly pipeline: 5 s timeout, retry with backoff and jitter for idempotent calls only, circuit breaker	M	After N 5xx the circuit opens; requests fail in under 50 ms
P5-5	PaymentAttempts and ProviderCallLogs with masked payloads and latency	M	No 13-19 digit number is ever persisted
P5-6	PaymentOrchestrator using two transactions: record attempt and set Processing, call provider, record outcome	L	No DB transaction is open during the HTTP call
P5-7	Confirm endpoint, admin sync (inquiry), attempts endpoint	M	tok_timeout returns pending_review in about 5 s, never failed
P5-8	Capture and refund through the provider consistent with ledger posting	M	Ledger is never posted without a confirmed provider action
P5-9	Tests: kill between transactions, 200 concurrent confirms with 20% injected errors; sequence diagrams (happy, decline, unknown outcome)	L	Every payment ends in exactly one legal state

Phase output

- A runnable simulator with a documented token-to-behaviour catalogue.
- Tables: PaymentAttempts, ProviderCallLogs.

- Endpoints: confirm, admin sync, attempts.
- Three sequence diagrams: happy path, decline path, unknown-outcome path.

Phase 6: Webhooks & Outbox (weeks 12-14)

Goal: accept provider events safely, and notify merchants reliably without losing or duplicating events.

ID	Ticket	Size	Done when
P6-1	Signature filter: raw body, HMAC-SHA256, constant-time compare, 5-minute timestamp window	M	Tampered body returns 401; 10-minute-old timestamp returns 400
P6-2	WebhookEvents with unique (Provider, ExternalEventId); store raw first, answer 200 fast	M	Same eventId sent 20 times in parallel gives one row
P6-3	InboundWebhookProcessor claiming rows with UPDLOCK, READPAST, ROWLOCK; out-of-order handling through the state machine	L	Captured arriving before authorized ends in the correct state
P6-4	OutboxMessages written in the same transaction as state changes	M	Rolled-back transaction leaves no outbox row
P6-5	OutboxDispatcher with leases (LockedUntil)	M	Two dispatchers never deliver the same message at once
P6-6	Webhook endpoint APIs: register, list, disable; secret returned once; SSRF guard	M	http://169.254.169.254/ and https://localhost are rejected
P6-7	MerchantWebhookDeliveryWorker: signed delivery, backoff 10 s, 1 m, 5 m, 30 m, 2 h, 12 h, then dead-letter; delivery log	L	A 500 endpoint follows the schedule then dead-letters
P6-8	Secret rotation with 24-hour grace, manual redeliver, GET /events polling fallback	M	Both old and new signatures verify during the grace window
P6-9	Event catalogue (payment., *refund.*) and tests: kill after commit before dispatch	L	Zero lost events after restart; inbound returns in under 100 ms

Phase output

- Tables: WebhookEvents, OutboxMessages, WebhookEndpoints, WebhookDeliveries.
- Public inbound webhook endpoint plus merchant webhook management and delivery history endpoints.
- Event catalogue document with versioned schemas.
- A sample merchant receiver that verifies signatures.

Phase 7: Reconciliation Engine (weeks 15-17)

Goal: prove FinCore's books match the provider's report and the bank, and explain every difference.

ID	Ticket	Size	Done when
P7-1	Simulator settlement report (CSV) with anomaly injection: missing, extra, altered, duplicated rows, wrong fees	M	Injected anomalies are configurable per date
P7-2	SettlementReportImporter: streaming CSV, row validation, SqlBulkCopy to staging, file-hash idempotency	L	Same file twice is a no-op; bad row is listed with its line number
P7-3	Bank statement import (CSV)	S	Lines stored and linked to batches
P7-4	ReconciliationEngine: exact match, then tolerant match (plus or minus 1 day), then classify leftovers, then batch total check; set-based SQL	L	1,000 payments on both sides give 1,000 matches and no discrepancies
P7-5	DiscrepancyDetector (pure rules) for MISSING_AT_PROVIDER, MISSING_INTERNALLY, AMOUNT_MISMATCH, STATUS_MISMATCH, DUPLICATE, FEE_MISMATCH, TIMING, BATCH_TOTAL_MISMATCH	M	Five injected anomalies reported exactly, no false positives
P7-6	SettlementPoster: Dr Bank, Dr Provider Fees, Cr Provider Clearing; payments become Settled; pending moves to available	L	Merchant pending to available equals settled net; trial balance still zero
P7-7	Run, discrepancy and settlement endpoints; resolve with an adjusting journal entry	M	Resolving twice returns 409; no UPDATE or DELETE on ledger
P7-8	Idempotent re-runs, TIMING classification at the day boundary, currency mismatch never matched	M	Re-running the same date creates no duplicate settlement entries

ID	Ticket	Size	Done when
P7-9	Daily report (JSON and CSV), discrepancy runbook, 1,000,000-row performance test	L	Under your time budget (for example 2 minutes) with flat memory

Phase output

- Tables: ProviderReportFiles, ProviderReportRows, BankStatementLines, ReconciliationRuns, ReconciliationMatches, Discrepancies, Settlements.
- Admin endpoints for imports, runs, discrepancies, resolution and settlements.
- A daily reconciliation report suitable for a finance team.
- A runbook: what to do for each discrepancy type.

Phase 8: Background Workers, Retries & Caching (weeks 18-19)

Goal: every asynchronous path survives crashes, and every stuck state has an owner that resolves it.

ID	Ticket	Size	Done when
P8-1	JobRunner base class: lease (60 s), retry policy, metrics, JobRuns logging, cancellation	L	Killed worker's lease expires and another finishes the work
P8-2	TimeProvider injected everywhere; fake-time retry tests	M	Retries at 10 s, 30 s, 90 s within jitter; 8th failure dead-letters
P8-3	DeadLetters table, list and requeue endpoints	M	Requeued item processes exactly once
P8-4	Leader election with sp_getapplock	M	Three instances: each job runs once per tick
P8-5	Workers: StuckPaymentSweeper, IdempotencyKeyCleanup, LedgerIntegrityJob, DailyReconciliationScheduler, DeadLetterMonitor	L	Sweeper resolves a payment stuck in Processing
P8-6	Graceful shutdown on SIGTERM	M	No half-posted journal entries

ID	Ticket	Size	Done when
P8-7	ICacheService with single-flight GetOrCreateAsync and cache decorators for API keys, currencies, fee schedules; explicit invalidation	M	500 concurrent misses cause one DB query; revoked key fails at once
P8-8	/health/live, /health/ready, admin jobs, trigger and cache invalidation endpoints	M	Ready turns unhealthy when DB is down or backlog is too high
P8-9	Tests: poison message, transient vs permanent errors, stale cache never used for balance or status decisions	L	Refund and capture read the database, verified with a deliberately stale cache

Phase output

- Tables: JobRuns, DeadLetters.
- All workers running as hosted services, separable into FinCore.Workers.
- A retry and backoff policy document and a "what may be cached" decision table.
- Admin job, dead-letter and health endpoints.

Phase 9: Security & Observability (weeks 20-22)

Goal: a merchant sees only its own data, every sensitive action is attributable, and you can ask new questions of the running system.

ID	Ticket	Size	Done when
P9-1	ApiKeys table with mode, scopes, roll with grace period and revoke	M	Revoked key rejected on the very next request
P9-2	Scope authorization	S	Key with only payments:read calling POST /payments gets 403 insufficient_scope
P9-3	EF global query filter by MerchantId and a test that fails if an entity lacks it	M	Another merchant's id returns 404 on every route
P9-4	Rate limiting per API key: token bucket, 429 with Retry-After	S	100 requests in 1 s: excess gets 429; other merchants unaffected

ID	Ticket	Size	Done when
P9-5	Staff authentication (JWT or OIDC) and RBAC roles	L	Support role cannot reverse ledger entries
P9-6	AuditSaveChangesInterceptor writing AuditLogs (insert-only)	M	Every admin state change produces one audit row
P9-7	Maker-checker approvals for reversal and write-off	M	Requester cannot approve; exactly one reversal after approval
P9-8	Serilog with redaction, strict JSON (reject unknown fields), security headers, request size limits	M	Logs contain zero API keys, secrets or 13-19 digit numbers
P9-9	OpenTelemetry traces (traceparent stored in outbox) and Prometheus metric catalogue	L	One trace links API, DB, outbox dispatch and merchant webhook
P9-10	Grafana dashboards, alert rules as code, threat model, OWASP API Top 10 checklist, dependency scan	M	Killing the simulator fires the provider error alert

Phase output

- Tables: ApiKeys (replacing the key hash on Merchants), AuditLogs, ApprovalRequests.
- Endpoints: API keys, staff token, audit logs, approvals, /metrics.
- Grafana dashboards and alert rules as code.
- A one-page threat model and an OWASP API Top 10 checklist mapped to controls.

Phase 10: Docker, CI/CD & Production Readiness (weeks 23-25)

Goal: one command starts the whole system, and a pipeline proves the golden invariants before anything ships.

ID	Ticket	Size	Done when
P10-1	Multi-stage Dockerfiles for api, workers, simulator: non-root, HEALTHCHECK	M	Containers run as non-root; image has no secrets

ID	Ticket	Size	Done when
P10-2	docker-compose with sqlserver, one-shot migrator, seq, prometheus, grafana, optional redis	M	docker compose up is healthy in under 5 minutes
P10-3	CI on every PR: build with warnings as errors, format check, tests, coverage gate (domain 90% or more), vulnerable-package scan, gitleaks, image scan	M	Failing test, coverage drop or High vulnerability blocks the pipeline
P10-4	Idempotent migration bundle and expand/contract discipline	M	Running the bundle twice is a no-op; old app version still works on new schema
P10-5	CD on merge: image tagged with git SHA, migrate staging, deploy, smoke test, k6, manual approval, production	M	/version shows the deployed git SHA
P10-6	/version endpoint; Swagger only in non-production	S	No secrets exposed
P10-7	k6 load scripts and a chaos script (DB blip, paused simulator, duplicate webhooks, restarts)	L	Targets met (for example 200 creates per second, p95 under 300 ms); zero invariant violations
P10-8	DayInTheLife test: 1,000 payments, 10% refunds, 5% declines, 2% timeouts, daily reconciliation	L	Debits equal credits; merchant balances equal settled nets; zero unexplained discrepancies
P10-9	Backup and restore drill	M	Restored DB passes /integrity; restore time recorded against the RTO
P10-10	README quick-start, C4 diagrams, ADRs, runbooks, thesis demo script	L	A stranger can run and understand the project from the README

Phase output

- A repository where docker compose up gives a working, observable FinCore stack.
- Green pipeline, published images, migration bundle artifacts.
- Load-test report (throughput, p50, p95, p99, error rate) and chaos-test report.
- Backup and restore drill record with measured RTO.

- Final demo script: onboard merchant, pay, timeout, webhook, refund, reconcile, resolve discrepancy, show trial balance and audit trail.